

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



ADVANCED DATA STRUCTURES AAT REPORT on

OPTIMISED DISJOINT SET UNION

Submitted by

Kashvi Agarwal (1BM23CS142)
Keerthi Reddy(1BM23CS137)
Kanishka Sharma(1BM23CS138)
Kashvi Agarwal(1BM23CS141)

Under the Guidance of
Prof. Rajeshwari K
Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February-May 2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visveswaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the AAT work entitled “**OPTIMISED DISJOINT SET UNION**” is carried out by **KASHVI AGARWAL(1BM23CS141),KEERTHI REDDY(1BM23CS137),KANISHKA SHARMA(1BM23CS138) and KASHVI AGARWAL (1BM23CS142)** who are bonafide students of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visveswaraya Technological University, Belgaum during the year 2024-2025. The AAT report has been approved as it satisfies the academic requirements in respect of **Advanced Data Structures (23CS6PEADS)** work prescribed for the said degree.

Signature of the Guide
Prof. Rajeshwari K
Assistant Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Kavitha Sooda
Prof.& Head, Dept. of CSE
BMSCE, Bengaluru

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **KASHVI AGARWAL(1BM23CS141),KEERTHI REDDY(1BM23CS137),KANISHKA SHARMA(1BM23CS138) and KASHVI AGARWAL (1BM23CS142)**, students of 6th Semester, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this AAT entitled "**OPTIMISED DISJOINT SET UNION**" has been carried out by us under the guidance of Prof. Namratha M, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester February to May 2025.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

Kashvi Agarwal (1BM23CS142)
Keerthi Reddy(1BM23CS137)
Kanishka Sharma(1BM23CS138)
Kashvi Agarwal(1BM23CS141)

TABLE OF CONTENTS

Chapter no.	Title	Page no.
1	PROBLEM STATEMENT	6
1.1	Motivation	6
1.2	Work highlights	7
2	DESIGN	8-9
3	ALGORITHM	10-11
4	RESEARCH PAPER AND DATA STRUCTURES USED	12-13
5	IMPLEMENTATION	
5.1	Code	14-19
5.2	Output	20-21
6	CONCLUSION	22-24

1. PROBLEM STATEMENT

In modern educational environments, student interactions extend beyond simple social friendships to include academic collaborations such as peer learning, doubt-solving, and subject-specific assistance. However, these relationships are often unstructured and difficult to analyze manually, especially in large student groups.

This project aims to model and analyze a hybrid student network that integrates both:

- Social relationships (friendships)
- Academic interactions (subject-wise help)

The primary objective is to efficiently identify student communities (friend groups) and analyze academic influence patterns within and across these communities.

To achieve this, the project uses the Disjoint Set Union (DSU) algorithm, optimized with path compression and union by rank, to detect connected components in the friendship graph. These components represent natural student communities.

On top of this, the system performs multiple analytical tasks such as:

- Identifying global and community-level influencers
- Detecting bridge students who connect different groups
- Comparing social connectivity vs academic contribution
- Predicting future influential students

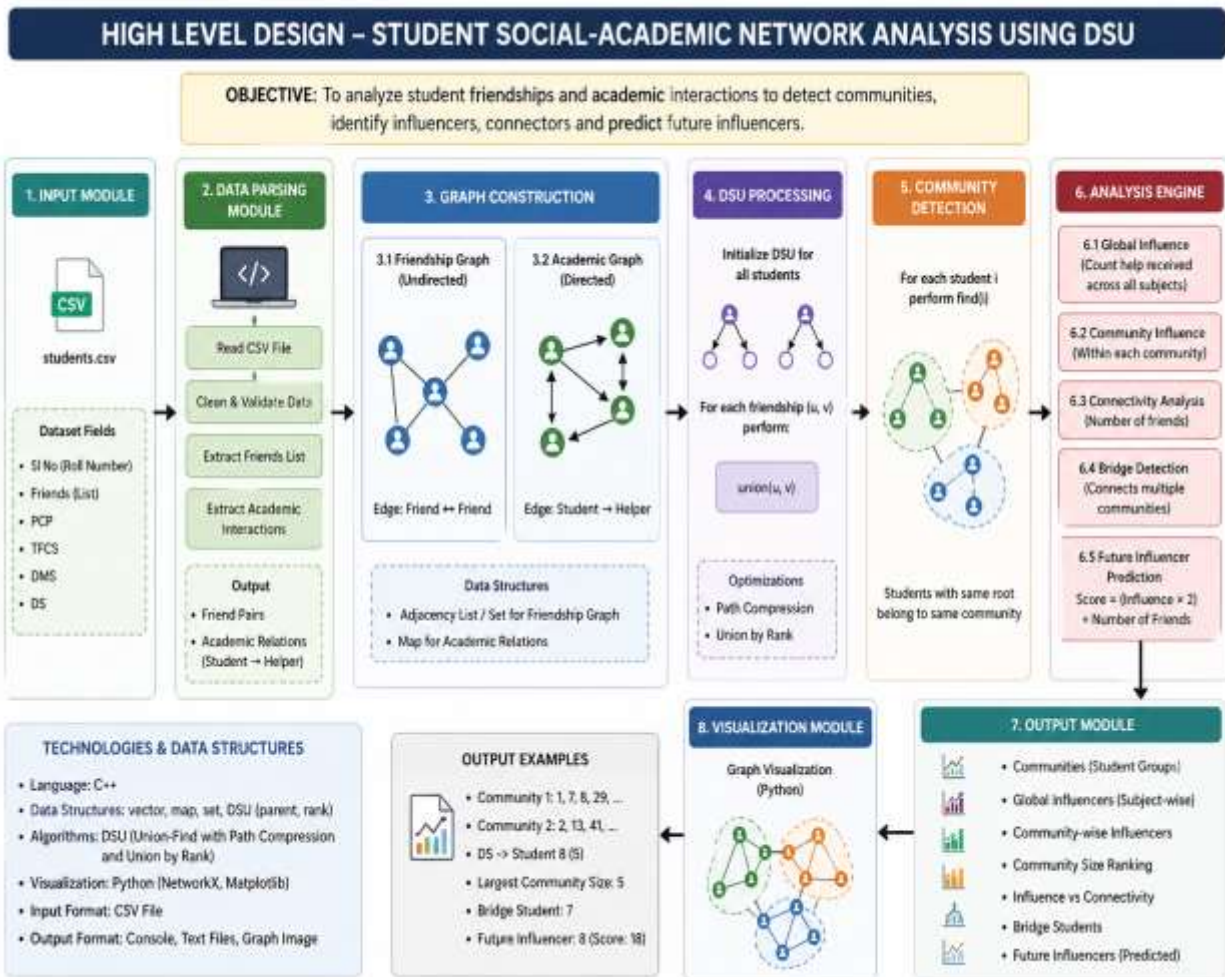
1.1 Motivation for choosing the problem statement.

In modern educational settings, student learning is highly influenced by both social interactions and academic collaborations. However, these relationships are often complex and not explicitly visible, making it difficult to identify influential students, effective study groups, and knowledge-sharing patterns. Traditional methods of analysis are inefficient when dealing with large datasets of student interactions. This motivated the need for a structured and scalable approach to model and analyze such networks. By using graph-based representation and the Disjoint Set Union (DSU) algorithm, the project provides an efficient way to detect communities and uncover hidden patterns in student behavior. The integration of social and academic data further enhances the relevance of the analysis, making it applicable to real-world educational environments.

1.2 Work highlights

This project involves the design and implementation of a hybrid student network model that combines both social and academic interactions. An optimized Disjoint Set Union (DSU) algorithm, incorporating path compression and union by rank, is used to efficiently identify communities within the friendship network. The system further analyzes academic interactions to determine global and community-level influencers based on how frequently students are consulted for help. Additional insights are generated by comparing social connectivity with academic contribution, identifying bridge students who connect different groups, and predicting future influencers using a scoring mechanism. The project also supports visualization of the network, enabling better interpretation of relationships and patterns. Overall, the work demonstrates an effective application of data structures and graph theory to solve a real-world problem in educational analytics.

2. DESIGN



The system is designed to analyze student interactions by integrating both social relationships and academic collaborations into a unified framework. Initially, the input is taken from a CSV dataset containing student details such as roll numbers, friendship lists, and subject-wise academic interactions. This data is processed in the parsing module, where it is cleaned and structured into meaningful formats, extracting friendship pairs and academic help relationships. Based on this processed data, two types of graphs are constructed: an undirected friendship graph representing social connections between students, and a directed academic graph representing help-seeking behavior.

The Disjoint Set Union (DSU) algorithm is then applied to the friendship graph to efficiently group students into communities using path compression and union by rank optimizations. Each group formed represents a connected component or a social community. Once the communities are identified, an analysis engine performs various computations, including identifying global and community-level influencers based on academic interactions, analyzing connectivity through the number of friendships, detecting bridge students who connect multiple

groups, and predicting future influencers using a scoring model. Finally, the results are displayed and optionally visualized using graph plotting techniques, providing a clear representation of student communities and interaction patterns. This modular design ensures scalability, efficiency, and ease of extension for future enhancements.

3. ALGORITHM FOR THE CHOSEN PROBLEM

Initialize DSU(n)

for each student i:

 parent[i] = i

 rank[i] = 0

for each friendship (u, v):

 union(u, v)

function find(x):

 if parent[x] != x:

 parent[x] = find(parent[x])

 return parent[x]

function union(u, v):

 rootU = find(u)

 rootV = find(v)

 if rootU != rootV:

 if rank[rootU] < rank[rootV]:

 parent[rootU] = rootV

 else if rank[rootU] > rank[rootV]:

 parent[rootV] = rootU

 else:

```
parent[rootV] = rootU
```

```
rank[rootU]++
```

```
for each student i:
```

```
    root = find(i)
```

```
    add i to community[root]
```

```
for each academic interaction (student → helper):
```

```
    increase influence[helper]
```

```
for each student:
```

```
    score = (influence * 2) + number_of_friends
```

Pseudocode along with necessary diagrams to show for a specific case of input and output sample.

4. RESEARCH PAPER AND DATA STRUCTURES USED

VPC: Pruning connected components using vector-based path compression for Graph500

Recent research in graph processing highlights the importance of efficient algorithms for handling large-scale networks. Bai et al. (2021) proposed an optimized method called Vector-based Path Compression (VPC) to improve the performance of connected component detection in massive graphs. Their work is based on enhancing the union-find (Disjoint Set Union) algorithm using advanced path compression techniques and optimized data structures. The study demonstrates that traditional approaches such as BFS and DFS are less efficient compared to union-find-based methods when dealing with large datasets. The proposed approach significantly improves performance by reducing traversal overhead and optimizing memory usage.

This research supports the use of Disjoint Set Union (DSU) in this project, as DSU provides an efficient way to detect connected components (student communities) with near constant time complexity. The concept of path compression used in the paper aligns with the optimization techniques applied in this system, making DSU highly suitable for analyzing large student interaction networks.

Data Structures Used

The implementation of the system utilizes several efficient data structures to ensure optimal performance and scalability.

- **Vector(vector):**
Used to store parent and rank arrays in DSU, as well as lists of students and edges. It provides fast access and dynamic resizing.
- **Map(map):**
Used to store academic influence data, where each student's help count is maintained. It allows efficient key-value mapping.
- **Set(set):**
Used to store unique community roots and avoid duplicates while identifying connected components.
- **Pair(pair<int,int>):**
Used to represent edges (friendship relationships) between students.

- **Disjoint Set Union Arrays (parent[], rank[]):**
Core structure used for grouping students into communities efficiently using path compression and union by rank.
- **Adjacency Representation (Optional Extension):**
Used for representing graph relationships, especially for visualization purposes.

5. IMPLEMENTATION

5.1 CODE

```
#include <bits/stdc++.h>

using namespace std;

// ===== DSU =====

class DSU {
    vector<int> parent, rankv;

public:
    DSU(int n) {
        parent.resize(n);
        rankv.resize(n, 0);
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // Path Compression
        return parent[x];
    }

    void unionSet(int x, int y) {
```

```

int rootX = find(x);
int rootY = find(y);

if (rootX == rootY) return;

if (rankv[rootX] < rankv[rootY])
    parent[rootX] = rootY;
else if (rankv[rootX] > rankv[rootY])
    parent[rootY] = rootX;
else {
    parent[rootY] = rootX;
    rankv[rootX]++;
}
}
};

// ===== Utility =====

vector<string> split(string s, char delim) {
    vector<string> tokens;
    string temp = "";
    for (char c : s) {
        if (c == delim) {
            if (!temp.empty()) tokens.push_back(temp);
            temp = "";
        } else temp += c;
    }
}

```

```

    }

    if (!temp.empty()) tokens.push_back(temp);

    return tokens;
}

// ===== MAIN =====

int main() {

    ifstream file("students.csv");

    if (!file) {
        cout << "Error: students.csv not found\n";
        return 0;
    }

    string line;
    getline(file, line); // skip header

    vector<vector<int>> friendsList;
    vector<vector<int>> academicHelp;
    int maxID = 0;

    // ===== READ CSV =====

    while (getline(file, line)) {

```

```

vector<string> row = split(line, ',');

int id = stoi(row[0]);
maxID = max(maxID, id);

// Friends parsing
vector<int> friends;
if (row[1] != "-") {
    vector<string> f = split(row[1], ' ');
    for (auto &x : f)
        friends.push_back(stoi(x));
}
friendsList.push_back(friends);

// Academic help parsing (PCP, TFCS, DMS, DS)
vector<int> help;
for (int i = 2; i < 6; i++) {
    if (row[i] != "-")
        help.push_back(stoi(row[i]));
}
academicHelp.push_back(help);
}

int n = maxID + 1;

```



```

// ===== DSU =====

DSU dsu(n);

// Apply union for friendships
for (int i = 0; i < friendsList.size(); i++) {
    int u = i + 1;
    for (int v : friendsList[i]) {
        dsu.unionSet(u, v);
    }
}

// ===== COMMUNITY DETECTION =====

map<int, vector<int>> communities;

for (int i = 1; i < n; i++) {
    int root = dsu.find(i);
    communities[root].push_back(i);
}

cout << "\n===== COMMUNITIES =====\n";

for (auto &c : communities) {
    cout << "Community: ";
    for (int x : c.second)
        cout << x << " ";
    cout << endl;
}

```

```

}

// ===== INFLUENCE =====

map<int, int> influence;

for (int i = 0; i < academicHelp.size(); i++) {
    for (int helper : academicHelp[i]) {
        influence[helper]++;
    }
}

cout << "\n===== GLOBAL INFLUENCE =====\n";

for (auto &p : influence) {
    cout << "Student " << p.first << " helped " << p.second << " times\n";
}

// ===== CONNECTIVITY =====

cout << "\n===== CONNECTIVITY =====\n";

map<int, int> connectivity;

for (int i = 0; i < friendsList.size(); i++) {
    connectivity[i + 1] = friendsList[i].size();
    cout << "Student " << i + 1 << " -> " << connectivity[i + 1] << " friends\n";
}

```

```

// ===== SCORE =====
cout << "\n===== FUTURE INFLUENCERS =====\n";

int bestStudent = -1, bestScore = -1;

for (int i = 1; i < n; i++) {
    int inf = influence[i];
    int conn = connectivity[i];
    int score = (inf * 2) + conn;

    cout << "Student " << i << " Score = " << score << endl;

    if (score > bestScore) {
        bestScore = score;
        bestStudent = i;
    }
}

cout << "\nTop Future Influencer: Student " << bestStudent << endl;

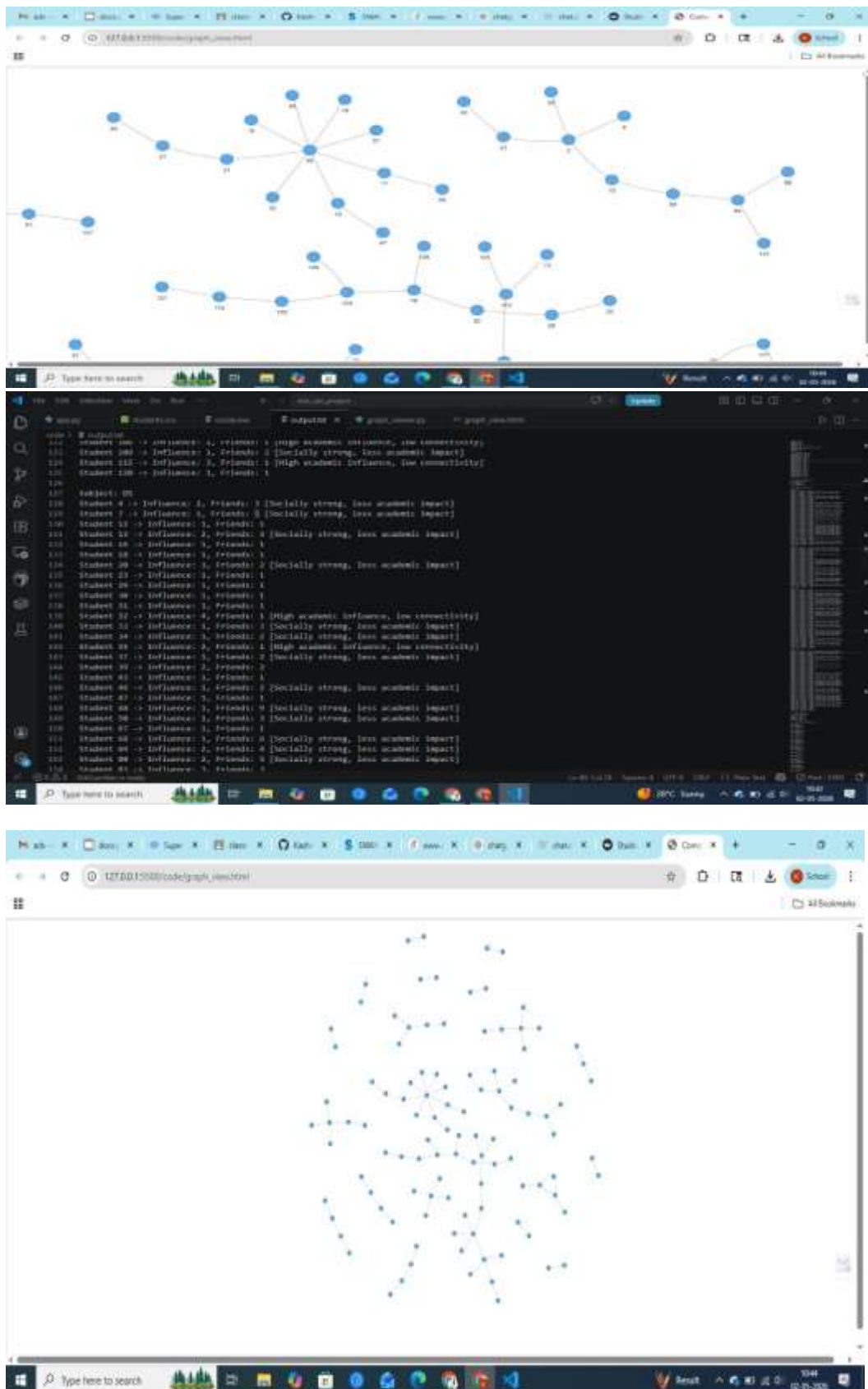
return 0;
}

```

5.2 OUTPUT

```
code > # output
47 Community Root 111
48 Community Root 116
49
50 ----- COMMUNITY SIZE RANKING -----
51 Community Root 76 -> Size: 18
52 Community Root 36 -> Size: 13
53 Community Root 100 -> Size: 10
54 Community Root 66 -> Size: 8
55 Community Root 78 -> Size: 6
56 Community Root 5 -> Size: 6
57 Community Root 66 -> Size: 5
58 Community Root 11 -> Size: 5
59 Community Root 1 -> Size: 4
60 Community Root 46 -> Size: 4
61 Community Root 40 -> Size: 4
62 Community Root 70 -> Size: 4
63 Community Root 8 -> Size: 3
64 Community Root 105 -> Size: 2
65 Community Root 64 -> Size: 2
66 Community Root 83 -> Size: 2
67 Community Root 27 -> Size: 2
68 Community Root 63 -> Size: 2
69 Community Root 79 -> Size: 2
70 Community Root 34 -> Size: 2
71 Community Root 34 -> Size: 2
72 Community Root 5 -> Size: 2
73
74 Largest Community -> Root 76 Size: 18
75
76 ----- GLOBAL TOP INFLUENCERS -----
77 DM -> Student 13 (4)
78 M -> Student 16 (4)
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
code > # output
40 Community Root 14 -> Size: 1
41 Community Root 14 -> Size: 1
42 Community Root 2 -> Size: 2
43
44 Largest Community -> Root 73 Size: 14
45
46 ----- GLOBAL TOP INFLUENCERS -----
47 DM -> Student 13 (4)
48 M -> Student 16 (4)
49 M -> Student 23 (3)
50 HCL -> Student 46 (4)
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72 ----- INFLUENCER VS CONNECTIVITY -----
73
74 Subject: DM
75 Student 1 -> Influence: 1, Friends: 1
76 Student 2 -> Influence: 1, Friends: 2 [Socially strong, less academic impact]
77 Student 4 -> Influence: 1, Friends: 2 [Socially strong, less academic impact]
78 Student 8 -> Influence: 1, Friends: 2
79 Student 10 -> Influence: 1, Friends: 1
80 Student 11 -> Influence: 1, Friends: 4 [Socially strong, less academic impact]
81 Student 15 -> Influence: 1, Friends: 3 [Socially strong, less academic impact]
82 Student 17 -> Influence: 1, Friends: 2 [Socially strong, less academic impact]
83 Student 18 -> Influence: 1, Friends: 1
84 Student 19 -> Influence: 1, Friends: 1
85 Student 20 -> Influence: 1, Friends: 2
86 Student 22 -> Influence: 1, Friends: 2 [High academic influence, low connectivity]
87 Student 23 -> Influence: 4, Friends: 1 [High academic influence, low connectivity]
88 Student 25 -> Influence: 1, Friends: 1
89 Student 26 -> Influence: 1, Friends: 1 [Socially strong, less academic impact]
90 Student 27 -> Influence: 1, Friends: 2 [High academic influence, low connectivity]
91 Student 29 -> Influence: 1, Friends: 1
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```



6. CONCLUSION

This project successfully demonstrates the application of **Disjoint Set Union (DSU)** and graph-based techniques to analyze student social and academic interactions. By modeling students as nodes and their relationships as edges, the system efficiently identifies communities using DSU with path compression and union by rank, ensuring near constant time performance even for larger datasets.

The integration of academic interaction analysis further enhances the system by identifying **global and community-level influencers**, as well as distinguishing between socially active and academically influential students. Additionally, the detection of **bridge students** and the implementation of a **scoring mechanism** for predicting future influencers provide deeper insights into student behavior and knowledge-sharing patterns.

Overall, the system offers a scalable, efficient, and practical approach for educational network analysis. It highlights how data structures and graph theory can be applied to solve real-world problems, making it highly relevant for applications such as peer learning optimization, mentoring systems, and educational data mining. Future enhancements can include weighted relationships, real-time analysis, and machine learning-based predictions to further improve accuracy and usability.

References:

Research Papers

1. Bai, H., Gan, X., Xu, T., Jia, M., Tan, W., Chen, J., & Zhang, Y. (2021). *VPC: Pruning connected components using vector-based path compression for Graph500*. **CCF Transactions on High Performance Computing**, 3(3), 271–285. <https://doi.org/10.1007/s42514-021-00070-z>
 2. Tarjan, R. E. (1975). *Efficiency of a Good But Not Linear Set Union Algorithm*. **Journal of the ACM**.
 3. Tarjan, R. E., & van Leeuwen, J. (1984). *Worst-Case Analysis of Set Union Algorithms*. **SIAM Journal on Computing**.
 4. Thota, S., Jain, M., Kamat, N., Malikireddy, S., Eranti, P. R., & Kuruvilla, A. (2020). *Union Find Shuffle: A Scalable Connected Components Algorithm*. <https://arxiv.org/abs/2012.05430>
-

Textbooks

5. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
 6. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson.
-

Web References

7. Disjoint Set Union (DSU) – CP Algorithms https://cp-algorithms.com/data_structures/disjoint_set_union.html
8. Union by Rank and Path Compression – GeeksforGeeks <https://www.geeksforgeeks.org/dsa/union-by-rank-and-path-compression-in-union-find-algorithm/>
9. Disjoint Set Data Structure – Wikipedia https://en.wikipedia.org/wiki/Disjoint-set_data_structure

In-Text Citations (Use inside report)

You can refer to the above sources in your report like this:

- The Disjoint Set Union (DSU) algorithm achieves near constant time complexity using path compression and union by rank (**Tarjan, 1975**).
- Advanced optimization techniques for connected component detection in large-scale graphs have been proposed using vector-based path compression (**Bai et al., 2021**).
- DSU is widely used for solving graph connectivity problems and is discussed in standard algorithm textbooks (**Cormen et al., 2009**).
- Practical implementations and explanations of DSU are available in algorithm resources such as CP Algorithms and GeeksforGeeks (**CP Algorithms, 2026; GeeksforGeeks, 2026**).